

Artificial Intelligence

Lecture 8 - Neural networks



- **Về kiến thức:**

- Hiểu rõ cấu tạo và nguyên lý hoạt động của một **Perceptron**.
- Giải thích được kiến trúc của mạng Multi-layer Perceptron (MLP).
- Nắm vững cơ chế lan truyền ngược (**Backpropagation**) và tối ưu hóa bằng **Gradient Descent**.

- **Về kỹ năng:**

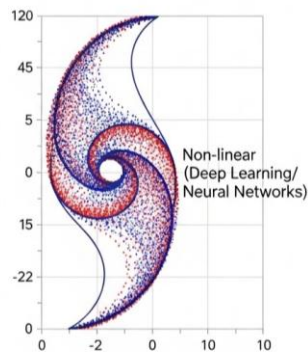
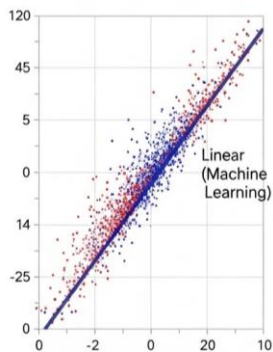
- Tính toán được đầu ra của mạng dựa trên bộ trọng số cho trước.
- Biết cách lựa chọn hàm kích hoạt (Activation Function) phù hợp cho từng bài toán.

- **Về tư duy:**

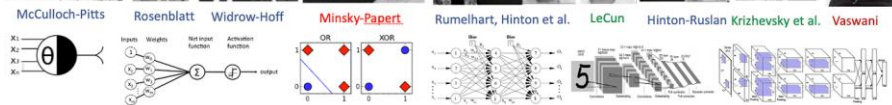
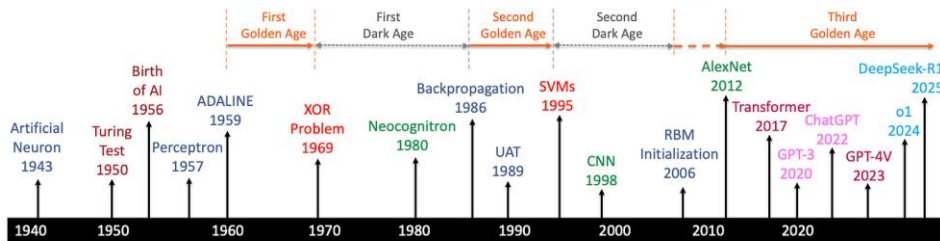
- Nhận diện được khi nào nên sử dụng Neural Network thay vì các thuật toán Machine Learning truyền thống.



- *Vượt qua giới hạn của các mô hình tuyến tính:*
 - **Bài toán Phi tuyến tính phức tạp:** Trong thực tế, ranh giới phân lớp thường không phải là đường thẳng hay mặt phẳng đơn giản.
 - **Khả năng tự trích xuất đặc trưng (Feature Engineering):**
 - ML truyền thống: Con người phải tự tìm ra các đặc trưng quan trọng (ví dụ: cạnh, màu sắc trong ảnh).
 - Neural Network: Các lớp ẩn (Hidden Layers) tự động học các đặc trưng từ mức độ thấp đến cao.
 - **Xử lý dữ liệu phi cấu trúc:** Hiệu quả vượt trội với Hình ảnh, Âm thanh và Văn bản.



A Brief History of AI with Deep Learning



"nghiên cứu các nhân tố ảnh hưởng"



[All](#) [News](#) [Videos](#) [Images](#) [More](#)

Tools

About 115,000 results (0.44 seconds)

<https://www.researchgate.net › 34110...> [Translate this page](#)

Nghiên cứu các nhân tố ảnh hưởng đến việc áp dụng chuẩn ...

Nghiên cứu các nhân tố ảnh hưởng đến việc áp dụng chuẩn mực kế toán việt nam của các doanh nghiệp nhỏ và vừa tại TP.HCM. April 2020. Authors:..

<https://www.researchgate.net › 32169...> [Translate this page](#)

(PDF) Nghiên cứu các nhân tố ảnh hưởng đến việc áp dụng ...

Nghiên cứu các nhân tố ảnh hưởng đến việc áp dụng chuẩn mực kế toán ở Việt Nam. January 2016. Authors: Dang Ngoc Hung at Hanoi University of Industry.

<https://js.vnu.edu.vn › article › view › PDF>

NGHIÊN CỨU Phân tích các nhân tố ảnh hưởng đến sự hài ...

by LT Dũng · 2013 · Cited by 2 — xã hội và nhân văn trên cơ sở nghiên cứu các nhân tố ảnh hưởng đến sự hài lòng của người sử dụng lao động đối với sản phẩm của quá trình đào tạo.

<http://tapchiconghuong.vn › bai-viet> [Translate this page](#)

Đề xuất mô hình nghiên cứu các nhân tố ảnh hưởng đến việc ...

May 10, 2021 — Bài viết nghiên cứu các lý thuyết liên quan và đề xuất mô hình nghiên cứu các nhân tố ảnh hưởng đến việc sử dụng TMDT của khách du lịch.

<https://tapchiconghuong.vn › bai-viet> [Translate this page](#)

Nghiên cứu các nhân tố ảnh hưởng đến ý định đi du lịch sau ...

Sep 23, 2021 — Nghiên cứu các nhân tố ảnh hưởng đến ý định đi du lịch sau đại dịch Covid-19 của người dân tại tỉnh Gia Lai. TCCTTHS.

<https://journalofscience.ou.edu.vn › ...> [Translate this page](#)

Nghiên cứu các nhân tố ảnh hưởng đến ý định khởi nghiệp ...

by VV Hiền · 2021 — Nghiên cứu các nhân tố ảnh hưởng tới tiềm năng khởi sự kinh doanh của sinh viên đại học [Research on factors influencing college students' entrepreneurial ...



simplylearn

What is Deep Learning?

The image contains several illustrative elements:

- A bowl of fruit connected by an arrow to a video camera, which is connected to a balance scale.
- A cloud containing a diagram of a neural network with nodes and connections.
- A cloud containing a diagram of a human brain with neural connections.
- A globe with a robot standing on top of it.
- A yellow warning triangle.
- A table titled "Features" with columns for "Tennis" and "Sunny", and rows for "Size" and "Type of tree".
- A diagram titled "Working of neural networks" showing three children looking at a board with the number 9 and a question mark.
- A diagram showing a vertical stack of nodes representing a neural network layer.

	Tennis	Sunny
Size		
Type of tree		

Working of neural networks

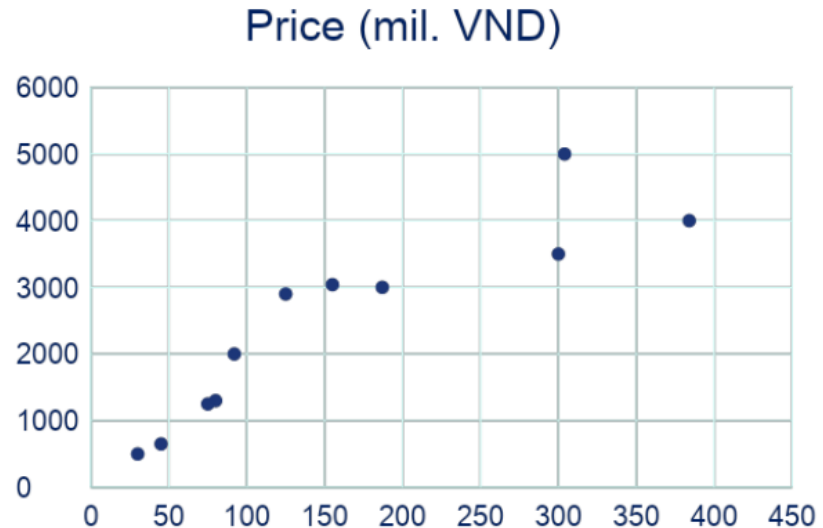
9 ? ?

Diagram showing a vertical stack of nodes representing a neural network layer.



Trở lại bài toán dự đoán giá nhà, cho bộ dữ liệu giả định:

Size (m ²)	Price (mil. VND)
30	500
45	650
80	1300
75	1250
125	2900
300	3500
187	3000
92	2000
384	4000
304	5000
155	3040

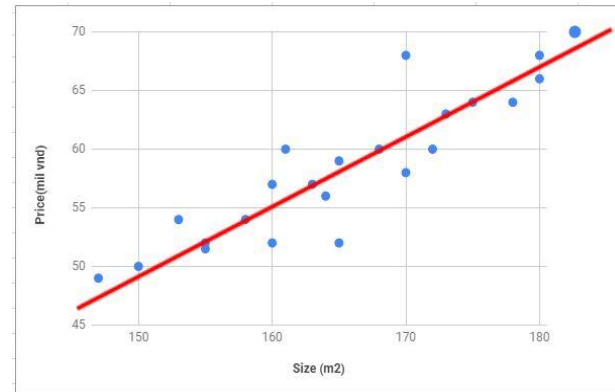
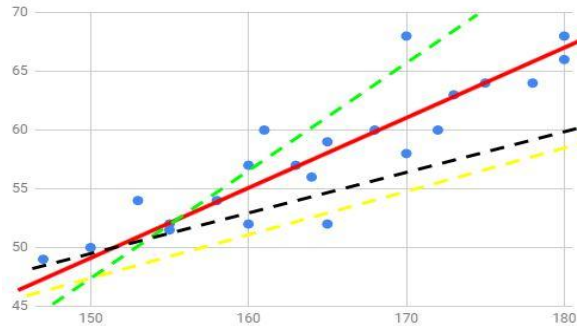




Linear regression is a linear model, e.g. a model that assumes a linear relationship between the input variables X and the single output variable y .

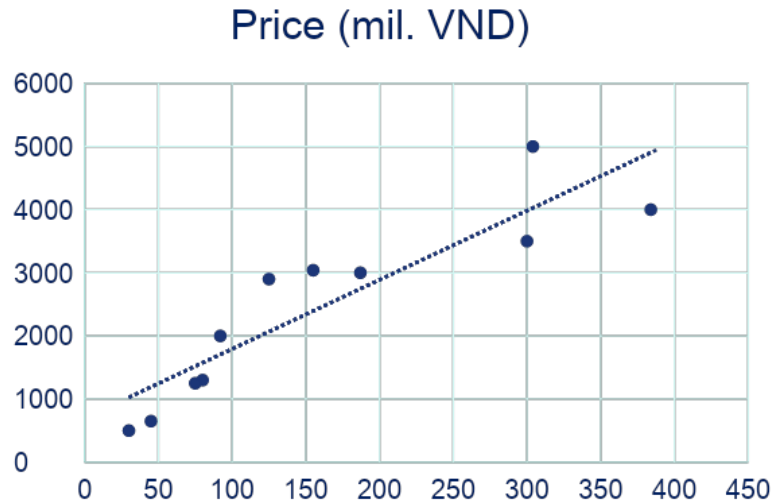
$$y = f(x) = b + w_1 x$$

How do we estimate the coefficients (“fit the model”)?



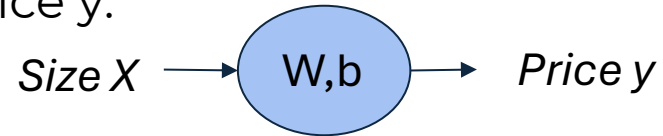


From the prepared dataset, we are able to draw a line to fit all the data points



This line is the simplest possible neural network

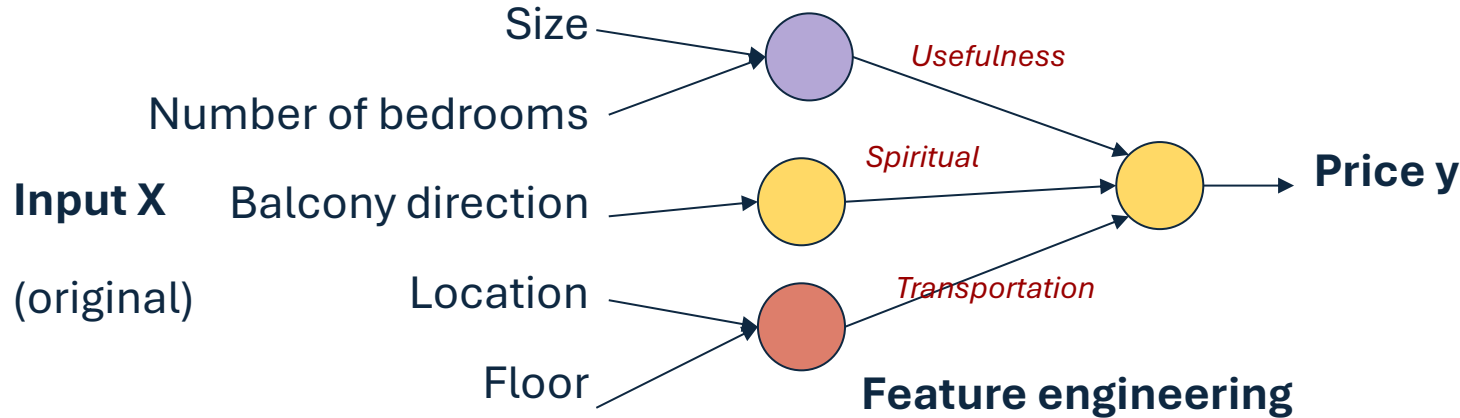
Input the size X , something in this circle shape will help you estimate the price y .



In term of AI, this circle shape is called a **neuron**, or **artificial neuron**, whose purpose is to compute the line we've drawn on the left



Let's make a bigger neural network



- The 1st neuron help us imagine how we design each part of the house
 - The 2nd neuron help us verify whether the direction is suitable to the owner
 - The 3rd neuron help us calculate the distance/time we have to spent to travel to work
- 3 features: ***usefulness, spiritual, transportation (hidden layers)***



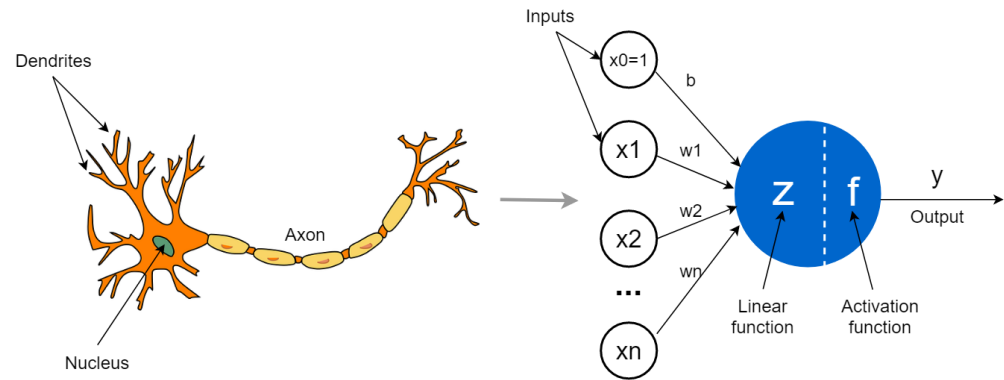
- Công thức tổng quát:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

- Trong đó:

- x: Các giá trị đầu vào (Features).
- w: Trọng số (Weights) - Tham số mạng cần học.
- b: Sai số chệch (Bias).
- f: Hàm kích hoạt (Activation Function).
- y: Kết quả đầu ra dự báo.

$$y = f(z)$$





- **Đầu vào (Inputs - x_i):** - Là các đặc trưng (features) của dữ liệu (ví dụ: diện tích nhà, số phòng).
- **Trọng số (Weights - w_i):** - Đại diện cho "tầm quan trọng" của tín hiệu đầu vào tương ứng.
 - Đây là những giá trị mà mô hình sẽ "học" và cập nhật trong quá trình huấn luyện.
- **Độ chệch (Bias - b):** - Là một hằng số cộng thêm vào tổng tín hiệu.
 - Giúp mô hình linh hoạt hơn trong việc dịch chuyển biên quyết định



- **Cơ chế:** Kết hợp tất cả inputs và weights thành một giá trị duy nhất.

- **Công thức đại số:**
$$z = x_1w_1 + x_2w_2 + \dots + x_nw_n + b = \sum_{i=1}^n x_iw_i + b$$

- **Dưới dạng Ma trận (Vectorization):**

- Gọi $X = [x_1, x_2, \dots, x_n]^T$ và $W = [w_1, w_2, \dots, w_n]^T$
- Công thức trở thành:

$$z = W^T X + b$$

- **Ý nghĩa:** Bước này thực hiện một phép biến đổi tuyến tính (Linear Transformation).



- Là hàm áp dụng lên kết quả của hàm tổng: $y = f(z)$
- **Tại sao cần tính Phi tuyến (Non-linearity)?**
 - Nếu không có hàm kích hoạt phi tuyến, việc chồng nhiều lớp neuron (Deep Network) cũng chỉ tương đương với một hàm tuyến tính đơn giản
 - Giúp mạng neuron có khả năng học và xấp xỉ các hàm số phức tạp trong thực tế.
- **Đặc điểm lý tưởng:**
 - Có đạo hàm (để phục vụ lan truyền ngược).
 - Tính toán nhanh chóng.



- **Step Function**

$$f(x) = \begin{cases} 1 & \text{if } x \geq \text{threshold} \\ 0 & \text{if } x < \text{threshold} \end{cases}$$

- Sử dụng trong mô hình Perceptron cổ điển

- **Sigmoid**

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Đầu ra nằm trong khoảng .
- Ưu điểm: Đạo hàm mượt, phù hợp cho bài toán xác suất.
- Nhược điểm: Dễ gây hiện tượng mất mát đạo hàm (Vanishing Gradient)



- **Tanh (Hyperbolic Tangent)**

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Đầu ra nằm trong khoảng , đối xứng qua trục 0.
- Thường hội tụ nhanh hơn Sigmoid

- **ReLU (Rectified Linear Unit)**

$$f(x) = \max(0, x)$$

- Ưu điểm: Tính toán cực nhanh, giảm thiểu Vanishing Gradient.

- **Softmax**

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

- Đặc điểm: Đầu ra là một vector xác suất (tổng bằng 1).
- Ứng dụng: Chuyên dùng cho lớp đầu ra của bài toán phân loại đa lớp (Multi-class Classification)



- Tìm hiểu các hàm kích hoạt khác: ***GELU, Swish, SoftPlus, Mish, Hard tanh, Hard Sigmoid***
 - Công thức
 - Trường hợp áp dụng

Bài tập:

- Cho một Neuron nhân tạo nhận 3 tín hiệu đầu vào:

Inputs: $x_1 = 0.5, x_2 = -1.2, x_3 = 0.8$

Weights: $w_1 = 0.4, w_2 = 0.5, w_3 = -0.2$

Bias: $b = 0.1$

- Tính giá trị đầu ra y trong 3 trường hợp sử dụng hàm kích hoạt khác nhau



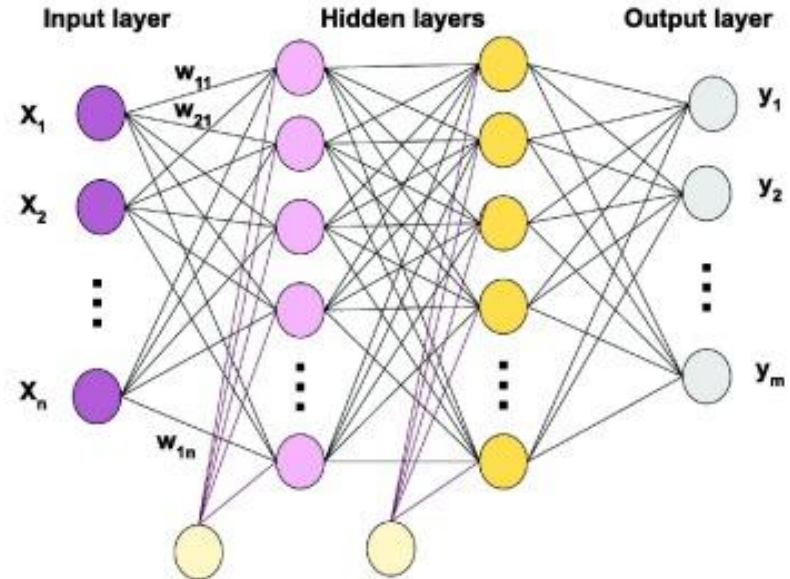
- **Bài toán XOR:** Perceptron chỉ có thể giải quyết các bài toán **phân tách tuyến tính** (linearly separable).
- **Minh họa:**
 - Hàm AND, OR: Có thể dùng 1 đường thẳng để chia tách các điểm dữ liệu.
 - Hàm XOR: Không thể dùng 1 đường thẳng duy nhất để tách các cặp khối .
- **Kết luận:** Để giải quyết các bài toán phi tuyến tính, chúng ta cần kết hợp nhiều Neuron lại với nhau tạo thành mạng đa lớp.



Cấu trúc tổng quát của một mạng Neural:

- **Lớp đầu vào (Input Layer):** Tiếp nhận dữ liệu thô.
- **Lớp ẩn (Hidden Layers):** Một hoặc nhiều lớp trung gian thực hiện trích xuất đặc trưng.
- **Lớp đầu ra (Output Layer):** Trả về kết quả dự báo cuối cùng.

Các neuron ở lớp trước kết nối đầy đủ (**Fully Connected**) với các neuron ở lớp sau. Mỗi kết nối có một bộ trọng số (W, b) riêng biệt.





- **Input Layer:**

- Số lượng neuron = Số lượng đặc trưng (features) của dữ liệu đầu vào. Không thực hiện tính toán, chỉ truyền giá trị đi.

- **Hidden Layers:**

- Là nơi thực hiện các phép biến đổi phi tuyến phức tạp.
- Được gọi là "ẩn" vì giá trị của chúng không có sẵn trong tập dữ liệu huấn luyện.
- Càng nhiều lớp ẩn và nhiều neuron, mạng càng có khả năng học các quy luật phức tạp.



- *Tùy biến theo mục tiêu bài toán:*

Bài toán	Số neuron đầu ra	Hàm kích hoạt (Activation)
Hồi quy (Regression)	1	Linear (hoặc không dùng hàm)
Phân loại nhị phân	1	Sigmoid
Phân loại đa lớp	(số lượng lớp)	Softmax
Phân loại đa nhãn	(số lượng lớp)	Sigmoid (cho từng đầu ra)



- Thay vì tính toán từng neuron đơn lẻ, ta gom tất cả trọng số của một lớp vào một ma trận.

- **Công thức cho một tầng (layer):**

$$Z^{(l)} = W^{(l)} \cdot A^{(l-1)} + b^{(l)}$$

$$A^{(l)} = f(Z^{(l)})$$

Trong đó:

- $W^{(l)}$: Ma trận trọng số của lớp
- $A^{(l-1)}$: Vector đầu ra của lớp trước đó.
- f : Hàm kích hoạt (áp dụng trên từng phần tử - element-wise).



Cách dữ liệu "chảy" qua mạng:

- Dữ liệu đầu vào X đi vào Input Layer.
- Tính tổng có trọng số và áp dụng hàm kích hoạt tại lớp ẩn đầu tiên.
- Kết quả được truyền tiếp làm đầu vào cho lớp ẩn tiếp theo.
- Lặp lại quá trình cho đến khi đạt tới Output Layer.
- Tính toán giá trị dự báo y.



- Khi gặp những bài toán phức tạp, liệu có thể tăng kích thước mạng theo chiều rộng (chỉ giữ 1 hidden layer, tăng số lượng neural) thay vì tăng độ sâu?
- Làm sao tìm ra bộ trọng số W và b tối ưu?



Bản chất của việc huấn luyện:

- **Mục tiêu:** Tìm ra bộ tham số (Weights và Bias) sao cho dự báo của mạng gần với giá trị thực tế nhất.
- **Học là quá trình tối ưu hóa:** - Ban đầu, và được khởi tạo ngẫu nhiên -> Sai số rất lớn.
 - Mạng cần một cơ chế để tự điều chỉnh và dựa trên sai số đó.
- **Công cụ:**
 - 1. **Hàm mất mát (Loss Function):** Đo lường sự sai khác.
 - 2. **Gradient Descent:** Thuật toán tìm cực tiểu của hàm mất mát.
 - 3. **Backpropagation:** Kỹ thuật tính toán các Gradient hiệu quả.



- **Bài toán Hồi quy (Regression):**

- Mean Squared Error (MSE): $L = \frac{1}{2}(\hat{y} - y)^2$

- **Bài toán Phân loại (Classification):**

- Binary Cross-Entropy (cho 2 lớp):

$$L = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$$

- Categorical Cross-Entropy (cho nhiều lớp).

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$



- Để giảm mất mát , ta cần đi ngược hướng với Gradient (đạo hàm).
- **Công thức cập nhật trọng số:**

$$W_{new} = W_{old} - \eta \cdot \frac{\partial L}{\partial W}$$

- Trong đó:

- η (learning rate): tốc độ học
- $\frac{\partial L}{\partial W}$: đạo hàm riêng của hàm Loss theo trọng số W.

$$b_{new} = b_{old} - \eta \cdot \frac{\partial L}{\partial b}$$

→ Trong mạng nhiều lớp, làm sao tính đạo hàm cho các lớp sâu bên trong?



Truy tìm "nguồn gốc" của sai số:

- **Ý tưởng:** Sai số xuất hiện ở lớp đầu ra (Output), sau đó được lan truyền ngược lại từng lớp phía trước để tính xem mỗi neuron đóng góp bao nhiêu vào sai số đó.
 - **Cơ chế:** Áp dụng quy tắc chuỗi (**Chain Rule**) trong đạo hàm để tính toán các Gradient từ lớp cuối cùng về lớp đầu tiên.
- Tránh việc phải tính toán lại các đạo hàm trùng lặp, tối ưu hóa tốc độ xử lý.



- Giả sử ta có hàm hợp: $y = f(g(x))$
- Đạo hàm của y theo x là: $\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}$
- Trong neural network:
 - Hàm Loss phụ thuộc đầu ra $\hat{y}$
 - $\hat{y}$ phụ thuộc tổng trọng số z
 - z phụ thuộc trọng số w

→ Công thức: $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$



- Giả sử bài toán Regression với

$$L = \frac{1}{2}(\hat{y} - y)^2$$

$$\hat{y} = \sigma(z)$$

- Đạo hàm của Loss theo đầu ra: $\frac{\partial L}{\partial \hat{y}} = (\hat{y} - y)$
- Đạo hàm của hàm kích hoạt (Sigmoid): $\frac{\partial \hat{y}}{\partial z} = \sigma(z)(1 - \sigma(z))$
- Đạo hàm của tổng theo trọng số: $\frac{\partial z}{\partial w} = x$

$$\rightarrow \frac{\partial L}{\partial w} = (\hat{y} - y) \cdot \sigma'(z) \cdot x$$



- Đặt δ là "sai số" tại một neuron:

$$\delta^{[L]} = \frac{\partial L}{\partial z^{[L]}} = (\hat{y} - y) \cdot f'(z^{[L]})$$

→ *Delta* cho biết một neuron cụ thể đóng góp bao nhiêu vào tổng sai số cuối cùng.

- **Công thức cập nhật:** $\frac{\partial L}{\partial w^{[L]}} = \delta^{[L]} \cdot a^{[L-1]}$

$$\frac{\partial L}{\partial b^{[L]}} = \delta^{[L]}$$



- Sai số tại lớp ẩn l được tính dựa trên sai số $l+1$ của lớp kết nối với nó:

$$\delta^{[l]} = (W^{[l+1]T} \cdot \delta^{[l+1]}) * f'(z^{[l]})$$

- **Quy luật:** Sai số được "phân phối" ngược lại dựa trên độ lớn của các trọng số kết nối. Trọng số càng lớn thì neuron đó càng phải chịu trách nhiệm nhiều cho sai số ở lớp sau.



- **Forward Pass:** Truyền dữ liệu vào, tính toán đầu ra cho từng lớp cho đến lớp cuối cùng.
 - **Compute Loss:** Tính sai số giữa đầu ra dự báo và thực tế.
 - **Backward Pass:** Tính toán các tín hiệu sai số *delta* và Gradient cho từng trọng số/độ chệch từ phải sang trái.
 - **Update Weights:** Sử dụng Gradient Descent để cập nhật W và b .
- Lặp lại quy trình này qua nhiều vòng (**Epochs**).



- **Nếu η quá nhỏ:** Mạng học rất chậm, tốn nhiều thời gian hội tụ.
 - **Nếu η quá lớn:** Mạng có thể bị dao động mạnh, không bao giờ đạt được cực tiểu, thậm chí làm hàm mất mát tăng dần (Exploding).
- **Kỹ thuật thực tế:** Thường bắt đầu với giá trị lớn (0.1, 0.01) và giảm dần theo thời gian (Learning Rate Decay).



- Cho 1 input: $(x = 1.0)$, 1 hidden layer (1 neural), 1 output
- Giá trị ban đầu khởi tạo:
 $w_1 = 0.5, b_1 = 0$
 $w_2 = 0.7, b_2 = 0$
- Mục tiêu thực tế: $y = 1.0$
- Hàm kích hoạt Sigmoid

Áp dụng quy trình 4 bước của Back propagation để tính giá trị cập nhật trọng số



• **Bước 1: Lan truyền tiến (Forward Propagation)**

- Tính giá trị tổng tại lớp ẩn: $z_1 = x \cdot w_1 + b_1$
- Tính giá trị kích hoạt lớp ẩn: $a_1 = \sigma(z_1)$
- Tính giá trị tổng tại lớp đầu ra: $z_2 = a_1 \cdot w_2 + b_2$
- Tính giá trị dự báo đầu ra: $\hat{y} = \sigma(z_2)$

• **Bước 2: Tính hàm mất mát (Loss Function)**

- Sử dụng công thức Mean Squared Error

$$L = \frac{1}{2}(\hat{y} - y)^2$$



• Bước 3: Lan truyền ngược (Backward Propagation)

- Tính các đạo hàm riêng (Gradient):

- Tại lớp đầu ra: $\delta_2 = \frac{\partial L}{\partial z_2} = (\hat{y} - y) \cdot \sigma'(z_2)$

(Lưu ý: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$)

$$\frac{\partial L}{\partial w_2} = \delta_2 \cdot a_1$$

$$\frac{\partial L}{\partial b_2} = \delta_2$$

- Tại lớp ẩn:

$$\delta_1 = \frac{\partial L}{\partial z_1} = (w_2 \cdot \delta_2) \cdot \sigma'(z_1)$$

$$\frac{\partial L}{\partial w_1} = \delta_1 \cdot x$$

$$\frac{\partial L}{\partial b_1} = \delta_1$$



- **Bước 4: Cập nhật trọng số (Weight Update)**

- Cập nhật theo công thức:

$$W_{new} = W_{old} - \eta \cdot \frac{\partial L}{\partial W}$$

- VD với $\eta = 0.2$:

$$w_{2_new} = w_2 - 0.2 \cdot \frac{\partial L}{\partial w_2}$$

$$b_{2_new} = b_2 - 0.2 \cdot \frac{\partial L}{\partial b_2}$$

$$w_{1_new} = w_1 - 0.2 \cdot \frac{\partial L}{\partial w_1}$$

$$b_{1_new} = b_1 - 0.2 \cdot \frac{\partial L}{\partial b_1}$$



- Cho 1 input: **$x=2.0$** , 1 hidden layer (1 neural), 1 output
- Giá trị ban đầu khởi tạo:
 $w_1 = 0.6$, bias $b_1 = 0$
 $w_2 = -0.4$, bias $b_2 = 0$
- Mục tiêu thực tế: **$y = 0.5$**
- Hàm kích hoạt Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$

Áp dụng quy trình 4 bước của Back propagation để tính giá trị cập nhật trọng số, thử với các giá trị $\eta = [0.1, 0.2]$



- Viết script/notebook Python để tự động hóa việc tính toán cho 2 bài tập trước đó.



Tại sao mạng sâu (Deep) lại khó học?

- **Nguyên nhân:** Đạo hàm của hàm Sigmoid có giá trị cực đại chỉ là 0.25.
- **Hiện tượng:** Khi nhân nhiều giá trị nhỏ (<1) với nhau qua nhiều lớp (Chain Rule), các Gradient ở lớp đầu tiên sẽ trở nên cực kỳ nhỏ (xấp xỉ 0).
- **Hậu quả:** Các lớp đầu tiên của mạng gần như không đổi trọng số $\rightarrow$ Mạng không học được gì.
- **Giải pháp:** Sử dụng hàm kích hoạt **ReLU** thay cho Sigmoid ở các lớp ẩn.



- **Tại sao không khởi tạo bằng 0?** Tất cả neuron trong cùng một lớp sẽ tính toán giống hệt nhau (Symmetry)
→ Mạng không học được các đặc trưng khác nhau.
- **Các phương pháp phổ biến:**
 - **Xavier Initialization:** Phù hợp cho hàm Sigmoid/Tanh. Giữ cho phương sai của tín hiệu ổn định qua các lớp.
 - **He Initialization:** hiệu quả khi sử dụng với hàm **ReLU**.
- **Mục tiêu:** Tránh hiện tượng tín hiệu bị triệt tiêu hoặc bùng nổ quá mức ngay từ đầu.



- **Stochastic Gradient Descent - SGD (Batch size = 1):**
Cập nhật trọng số sau mỗi mẫu dữ liệu. Rất nhanh nhưng dao động cực lớn.
- **Batch (Batch size = Toàn bộ tập dữ liệu):** Tính trung bình sai số của toàn bộ tập dữ liệu rồi mới cập nhật trọng số 1 lần.

→ *Rất ổn định nhưng chậm và tốn bộ nhớ (RAM/VRAM).*

- **Mini-batch (Batch size = 32, 64, 128...):**
 - Tận dụng sức mạnh tính toán song song của GPU.
 - Giúp hội tụ nhanh và ổn định hơn.

Tại sao batch-size thường chọn là số mũ của 2?



- **Vấn đề của SGD:** Dễ bị kẹt tại cực tiểu địa phương.
- **Các cải tiến:**
 - **Momentum:** Cộng thêm "đà" từ các bước trước đó để vượt qua các vùng bằng phẳng.
 - **RMSprop:** Tự động điều chỉnh tốc độ học cho từng tham số.
 - **Adam (Adaptive Moment Estimation):** Kết hợp Momentum và RMSprop. Hiện là bộ tối ưu hóa phổ biến nhất vì tính ổn định và hiệu quả.



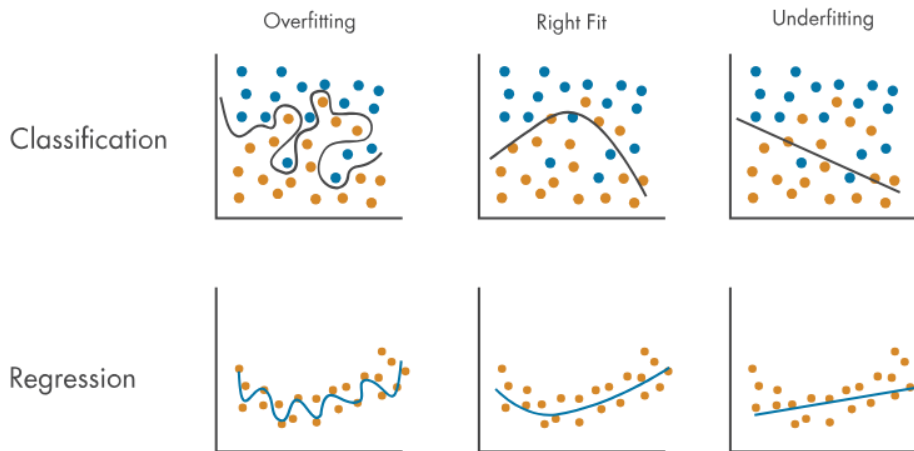
- Viết script/notebook Python minh họa việc huấn luyện mạng neural với Mini-batch và tối ưu tham số mà chỉ sử dụng các thư viện tính toán thông thường như Numpy
- Update script/notebook có sử dụng Tensorflow / PyTorch



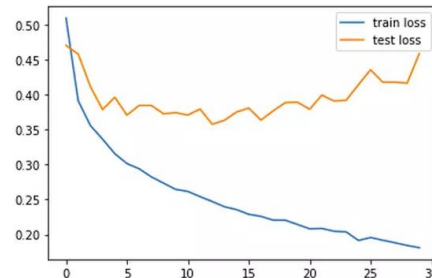
- **Tại sao cần chuẩn hóa?** Nếu các đặc trưng có thang đo quá khác biệt (ví dụ: tuổi 0-100 vs. thu nhập 0-1 tỷ), mặt cầu Loss sẽ bị kéo dẫn, khiến Gradient Descent khó hội tụ.
- **Các phương pháp:**
 - **Scaling:** Đưa dữ liệu về khoảng .
 - **Standardization (Z-score):** Đưa về phân phối chuẩn có và .
- **Batch Normalization:** Kỹ thuật chuẩn hóa dữ liệu ngay bên trong các lớp ẩn của mạng để tăng tốc độ huấn luyện.



- **Hiện tượng:** Mạng học quá tốt trên tập Training (Loss cực thấp) nhưng dự báo sai lệch lớn trên tập Validation/Test.



! Cách nhận biết: Đồ thị Training Loss tiếp tục giảm trong khi Validation Loss bắt đầu tăng ngược lại.





- **Ý tưởng:** Thêm một "hình phạt" (penalty) vào hàm mất mát dựa trên độ lớn của trọng số.
 - **L2 Regularization (Weight Decay)**

$$L_{total} = L + \frac{\lambda}{2} \sum ||W||^2$$

- **L1 Regularization:**

$$L_{total} = L + \lambda \sum |W|$$

- Tác dụng:
 - Ngăn cản trọng số trở nên quá lớn.
 - Giảm thiểu sự phụ thuộc quá mức vào vài đặc trưng đơn lẻ.



- **Cơ chế:** Trong mỗi bước huấn luyện, ta ngẫu nhiên "tắt" (set giá trị về 0) một tỷ lệ neuron nhất định (ví dụ 20% - 50%) ở các lớp ẩn.
- **Tại sao nó hiệu quả?**
 - Ngăn chặn các neuron quá phụ thuộc vào nhau (Co-adaptation).
 - Giống như việc huấn luyện một "tập thể" các mạng nhỏ khác nhau, sau đó kết hợp lại.
- **Lưu ý:** Chỉ sử dụng Dropout khi **Huấn luyện**, không dùng khi **Dự báo (Inference)**.